ALEXANDRE Ewan, Groupe 1
05/11/26

TP01 Installation et compréhension de React

Table des matières

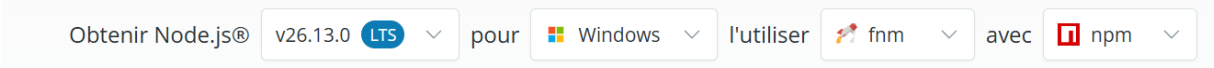
1) Problématique..... 1

1) Problématique

Nous souhaitons installer et comprendre comment utiliser la bibliothèque JavaScript **React** pour réaliser des interfaces utilisateurs plus avancées.

2) Prérequis

Afin de pouvoir installer React, nous devons d'abord installer Node.js en se dirigeant sur le lien suivant : https://nodejs.org/fr/download et en choisissant la dernière version de Node en version LTS :



Ensuite en dessous on peut soit utiliser les commandes qui nous sont données ou utiliser l'installateur fourni :

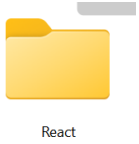


Enfin, on peut vérifier l'installation avec la commande suivante :

```
C:\Users\Ewan Alexandre>node -v
v18.20.7
```

La version est affichée donc Node.js a bien été installé.

Lorsque tout est bon, on peut déjà commencé par créer un dossier pour le projet React :



Et ensuite on se place dessus sur cmd puis on utilise la commande suivante pour initialiser le projet :

```
C:\Users\Ewan Alexandre\Videos\React>npm create vite@latest tickets-app -- --template react

> npx
> cva tickets-app --template react


create-vite v9.3.4

|
o  Which linter to use?
|  Oxlint
|
o  Install with npm and start now?
|  Yes
|
o  Scaffolding project in C:\Users\Ewan Alexandre\Videos\React\tickets-app...
|
o  Installing dependencies with npm...
||
```

Lorsqu'on met **Yes** cela exécute automatiquement les tâches 2,3,4, qui consiste à se placer dans le dossier tickets-app, à installer React et à lancer le serveur local.

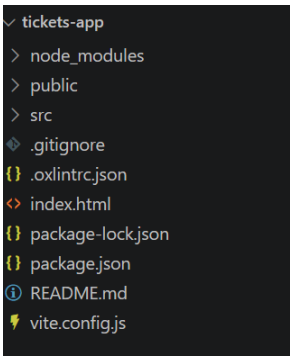
Vite c'est un outil de développement qui permet de créer, configurer et lancer facilement des applications web modernes, notamment avec React.

3) Exercices

Exercice 1 :

Nous allons maintenant ouvrir le projet dans VSCode, pour ça on fait d'abord Ctrl+C pour arrêter le serveur local puis on utilise la commande suivante : `C:\Users\Ewan Alexandre\Videos\React>code tickets-app`

Ensuite, on peut explorer l'arborescence et voir la fonction de chaque fichier/dossier :



Voici le rôle de tous les fichiers et dossiers principaux :

Fichier	**Rôle**
Package.json	Ce fichier contient les dépendances, les informations et les scripts du projet
Node_modules/	Ce dossier contient les bibliothèques installées avec npm
Index.html	Page HTML de l'application
Src/main.jsx	Point d'entrée de l'application React
Src/App.jsx	Composant principal de l'application
Src/assets/	Sert à contenir les ressources comme les images ou les vidéos.

Dans src/main.jsx nous pouvons voir ceci :

```
createRoot(document.getElementById('root')).render(
```

Cette instruction récupère l'élément HTML ayant pour identifiant « root » et affiche l'application React grâce à la commande render().

Exercice 2 :

On se place sur App.jsx et remplaçons le contenu par le code donné sur le site du BTS :

App.jsx

```jsx
function App() {
  return (
    <div>
      <h1>Gestion des tickets</h1>
      <p>Bienvenue sur l'application WebDev Solutions.</p>
    </div>
  );
}

export default App;
```

Ensuite on efface tous le contenu de chaque fichier .css.

Dans le dossier src nous allons créer un dossier **components** dans lequel nous créons un fichier Header.jsx qui contiendra le code donné sur le site :



Ensuite on modifie App.jsx pour importer la fonction Header :

```jsx
import Header from './components/Header';

function App() {
  return (
    <div>
      <Header />
      <p>Bienvenue sur l'application WebDev Solutions.</p>
    </div>
  );
}

export default App;
```

Le JSX est une extension de JavaScript qui permet d'écrire une syntaxe ressemblant au HTML directement dans le code JavaScript afin de décrire l'interface d'une application React.

Il y a tout de même trois différences avec le HTML :

- Au lieu de class on utilise className

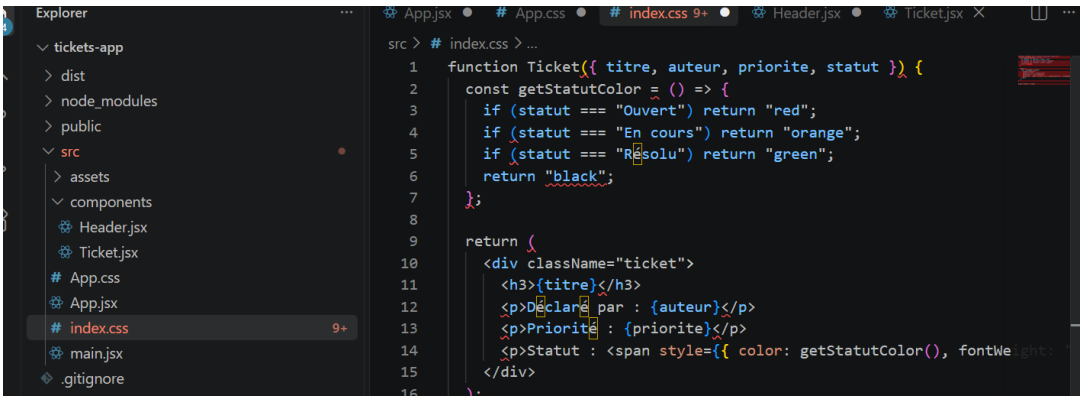
- Les expressions JavaScript sont intégrées avec des accolades {}.

- Les balises doivent être correctement fermées, même les balises simples comme <img />.

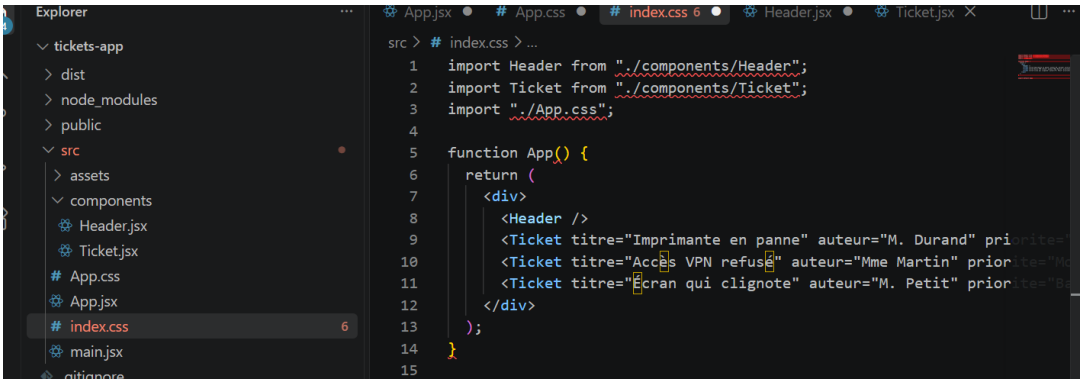
React doit recevoir un seul élément parent pour organiser les éléments retournés par un composant.

Exercice 3 :

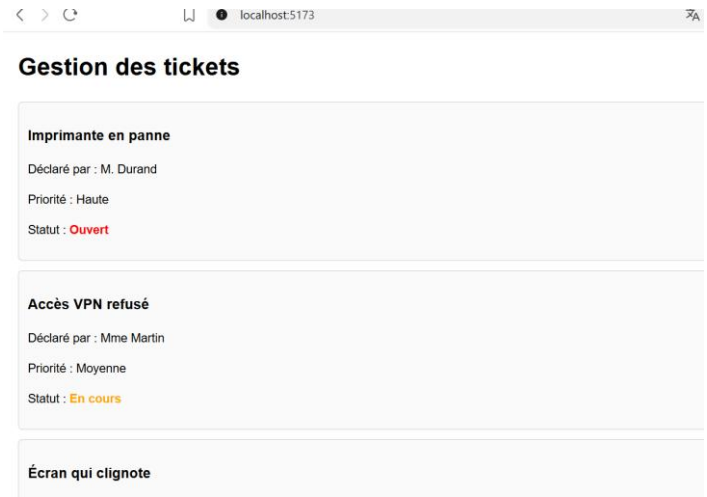
On crée le composant Ticket.jsx dans le dossier src/components/ pour afficher un ticket avec ses informations. On ajoute une prop statut qui permet d'afficher la couleur du statut en rouge si « Ouvert », en orange si « En cours » et en vert si « Résolu » :



Ensuite dans App.jsx, on importe le composant Ticket et on l'utilise avec au moins trois tickets différents :



Voici l'affichage obtenu dans le navigateur avec les couleurs selon le statut :

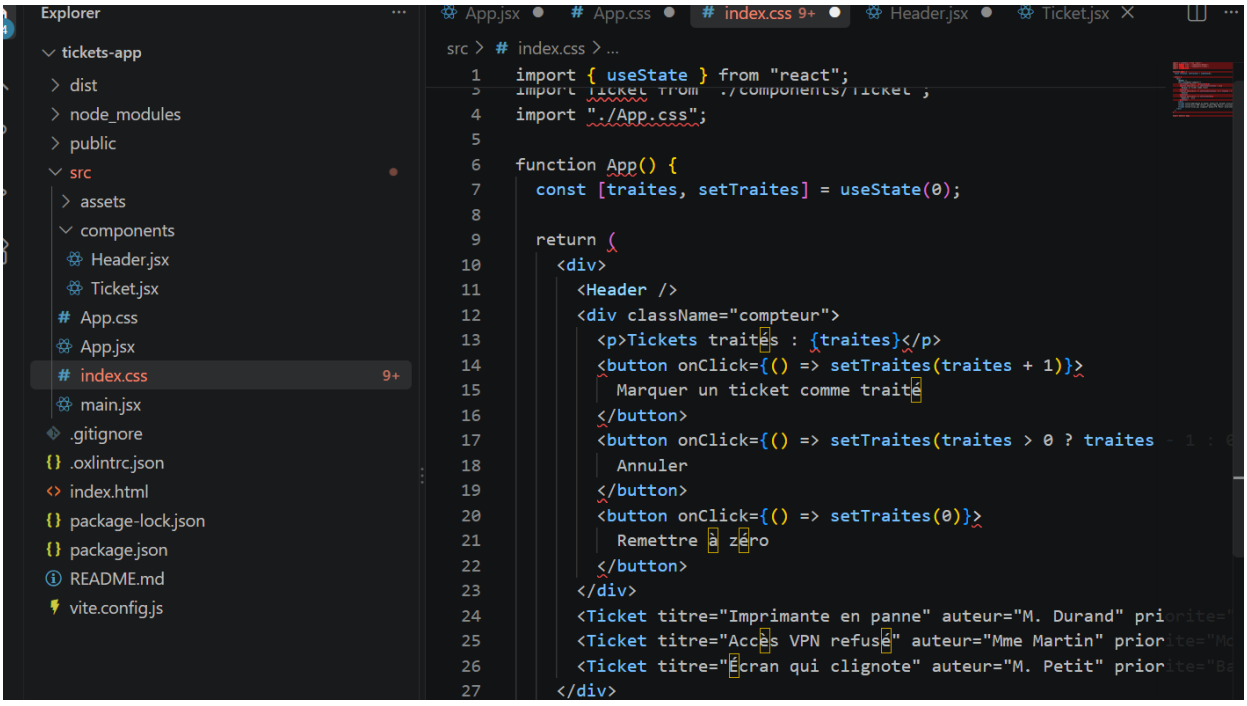


Question 7 : Les props sont-elles modifiables par le composant qui les reçoit ? Justifiez.

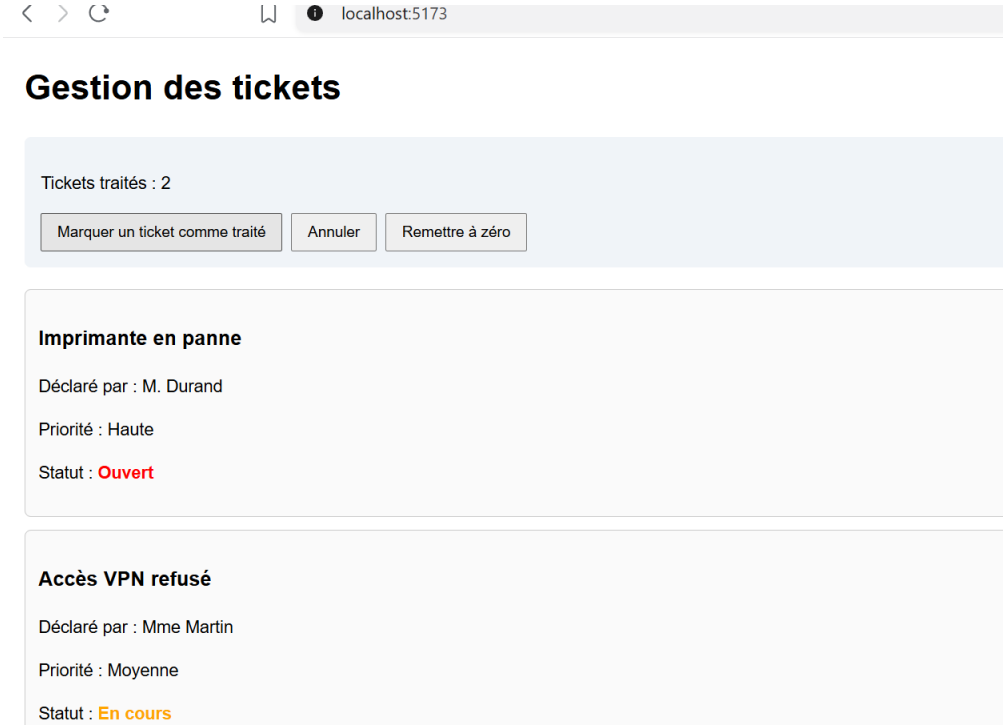
Non, les props ne sont pas modifiables par le composant qui les reçoit, elles sont en lecture seule (immuables). En React, le flux de données est unidirectionnel (du composant parent vers le composant enfant). Si une donnée doit changer au cours du temps, il faut utiliser un état (state) géré par le composant ou son parent.

Exercice 4 :

On importe le hook useState dans App.jsx pour gérer un compteur de tickets traités. On ajoute le bouton pour marquer un ticket comme traité (+1), un bouton pour réinitialiser le compteur à zéro et un bouton pour annuler qui empêche le compteur de descendre en dessous de zéro avec une condition :



Voici le résultat dans le navigateur avec le compteur interactif et les trois boutons :



Question 8 : Quelle est la différence entre une prop et un state ?

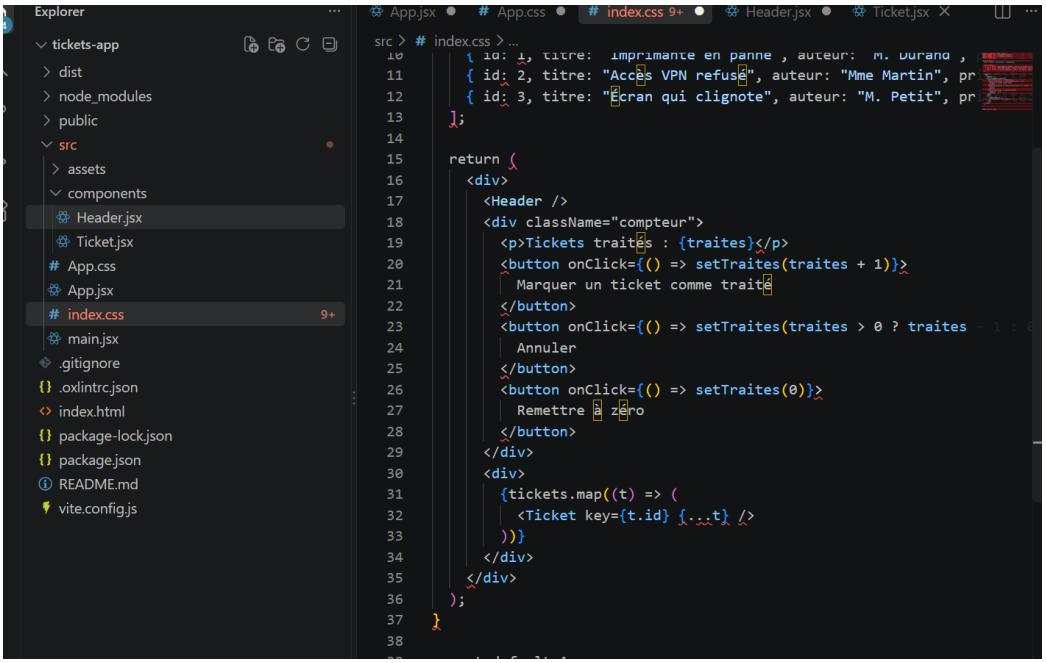
Une prop est une donnée transmise de l'extérieur par un composant parent à un composant enfant ; elle est en lecture seule pour l'enfant. Un state est une donnée interne et dynamique propre au composant, gérée par useState, qui peut être modifiée à l'aide de sa fonction de mise à jour (comme setTraites) et dont la modification provoque un nouveau rendu (re-render) de l'interface.

Question 9 : Pourquoi ne faut-il pas écrire traites = traites + 1 directement ?

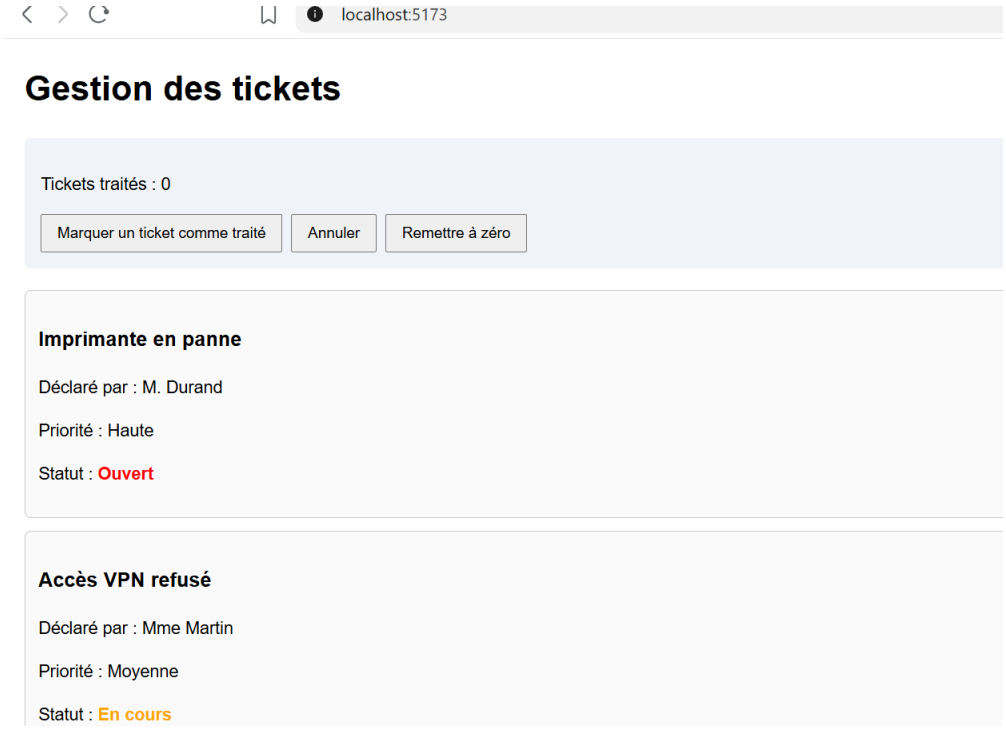
Parce que React ne détecte pas les modifications directes de variables. Si on modifie traites directement, la valeur en mémoire change mais React ne déclenche pas de nouveau rendu à l'écran, donc l'interface n'est pas mise à jour. Il faut impérativement appeler setTraites() pour que React prenne en compte le nouvel état et rafraîchisse la vue.

Exercice 5 :

On remplace les balises écrites à la main par un tableau d'objets JavaScript tickets et on utilise la méthode .map() pour générer la liste dynamiquement :



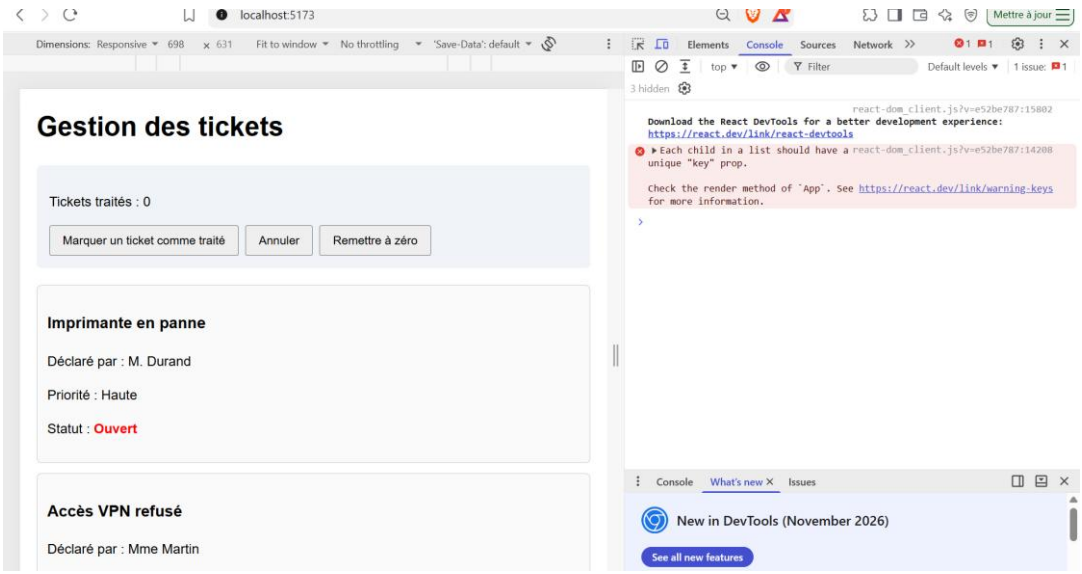
Voici l'affichage des tickets générés dynamiquement dans le navigateur :



Question 10 : À quoi sert la prop key ? Que se passe-t-il si on l'oublie (console du navigateur) ?

La prop key permet à React d'identifier de façon unique et stable chaque élément d'une liste dans le DOM virtuel. Cela lui permet de savoir exactement quels éléments ont été ajoutés, modifiés ou supprimés lors d'une mise à jour sans avoir à reconstruire toute la liste.

Si on oublie la prop key, React affiche un avertissement dans la console du navigateur (Warning: Each child in a list should have a unique 'key' prop) :



4) Conclusion

En conclusion, ce TP m'a permis d'apprendre à mettre en place un projet React avec l'outil Vite, de comprendre la structure des composants en JSX, de faire circuler des données avec les props et de gérer l'interactivité d'une application web grâce au hook useState et au rendu de listes dynamiques.